VesselVerse

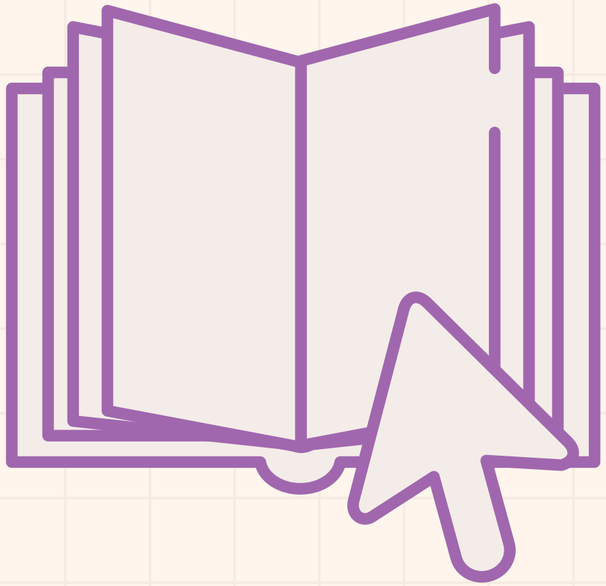
A Dataset and Collaborative Framework for Vessel Annotation

IMPROVING GIT & DVC WORKFLOWS FOR COLLABORATIVE RESEARCH

Alice Boccadifuoco | February 2026 | EURECOM – Semester Project

TABLE OF CONTENTS

1. Introduction
2. The Problem
3. Solution Overview
4. Repository Architecture
5. DVC + Git Integration
6. Python Scripts
7. Collaboration Flow
8. Impact & Results
9. Future Work
10. Main Documentation



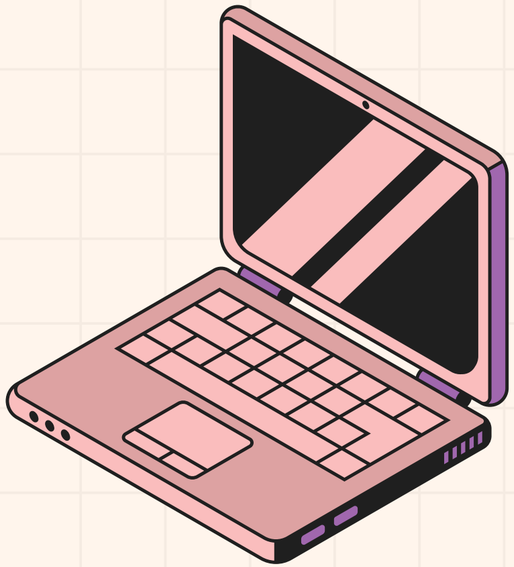
1. **INTRODUCTION -** WHAT IS VESSELVERSE?

VesselVerse: A Collaborative Medical Imaging Platform

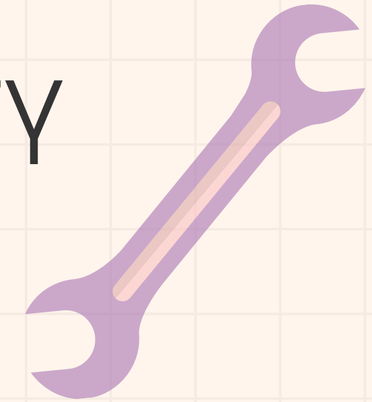
🧠 **Brain vessel segmentation datasets**
🔬 **Multiple annotation models (nnUNet, STAPLE, Expert annotations)**
👥 **Multi-user collaboration environment**
📊 **Large-scale data management (100s of GB)**

Technologies:

- **Git: Code versioning**
- **DVC (Data Version Control): Large file management**
- **Google Drive: Remote storage backend**



2. **THE PROBLEM** MANUAL COMPLEXITY

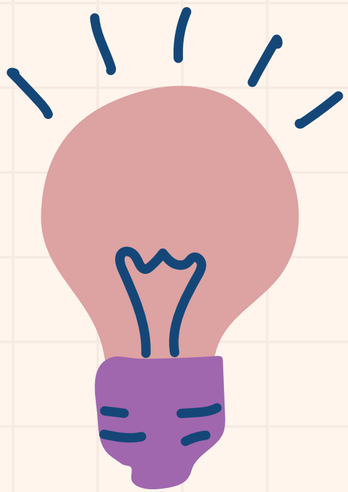


- **Manual Complexity**

```
# Users had to run 15+ commands manually:
cd datasets/D-IXI
dvc init
dvc remote add storage gdrive://...
dvc remote modify storage gdrive_service_account credentials.json
dvc add IXI_TOT/
git add IXI_TOT.dvc .gitignore
git commit -m "Add IXI data"
dvc push
git push
# ...and repeat for each dataset!
```

- **Unclear Repository Structure**
  - Framework and Dataset code mixed together
  - Confusion: "Where do I put my data?"
  - "What's tracked by Git vs DVC?"
- **High Barrier to Entry**

3. SOLUTION OVERVIEW

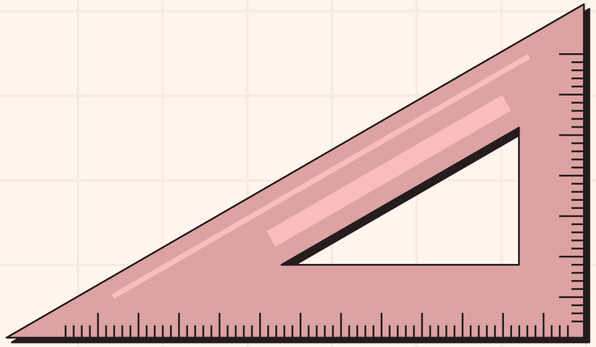


- **Automated CLI Scripts**
  - **vesselverseOwner.py –** Dataset management for maintainers
  - **vesselverseUser.py –** Simplified interface for contributors
  - **vv_utils.py –** Shared utilities
- **Clear Repository Architecture**

```
vesselverse/                    ← Parent workspace
├── vesselverseOwner.py         ← Scripts at root
├── vesselverseUser.py
├── credentials/
├── VesselVerse-Framework/      ← Functions & tools ONLY
└── VesselVerse-Dataset/        ← Data management ONLY
```

- **Integrated Pull Request Workflow**
  - One command: Create PR from terminal
  - Automated branch creation & upstream detection

4. REPOSITORY ARCHITECTURE



```
vesselverse/                        ✅ Clear structure
│
├── vesselverseOwner.py             📍 Scripts at root level
├── vesselverseUser.py
├── vv_utils.py
├── config.py
├── credentials/                    🔐 Credentials (git-ignored)
│
├── VesselVerse-Framework/          🛠️ FUNCTIONS & TOOLS
│   ├── src/                        • 3D Slicer extension
│   ├── notebooks/                  • Analysis notebooks
│   ├── scripts_py/                 • Utility scripts
│   └── docs/                       • Documentation
│
└── VesselVerse-Dataset/            📊 DATA MANAGEMENT
    └── datasets/                   • DVC-tracked data
        ├── D-IXI/                  • Independent DVC configs
        ├── D-COW23MR/              • Google Drive remotes
        ├── D-ITKTubeTK/
        └── D-Expert/               • User contributions
```

Separation:
Framework(code) vs Dataset (data)

Independence:
Each dataset has its own
DVC configuration

Clarity:
Python scripts orchestrate
both repositories
from parent folder

Full Python migration

5. **DVC + GIT INTEGRATION**

WHY NOT JUST GIT?

Git is NOT designed for large files:

- Slow and bloated repositories
- File size limits (GitHub: 100 MB)
- Clone/pull become impractical

VesselVerse Solution:

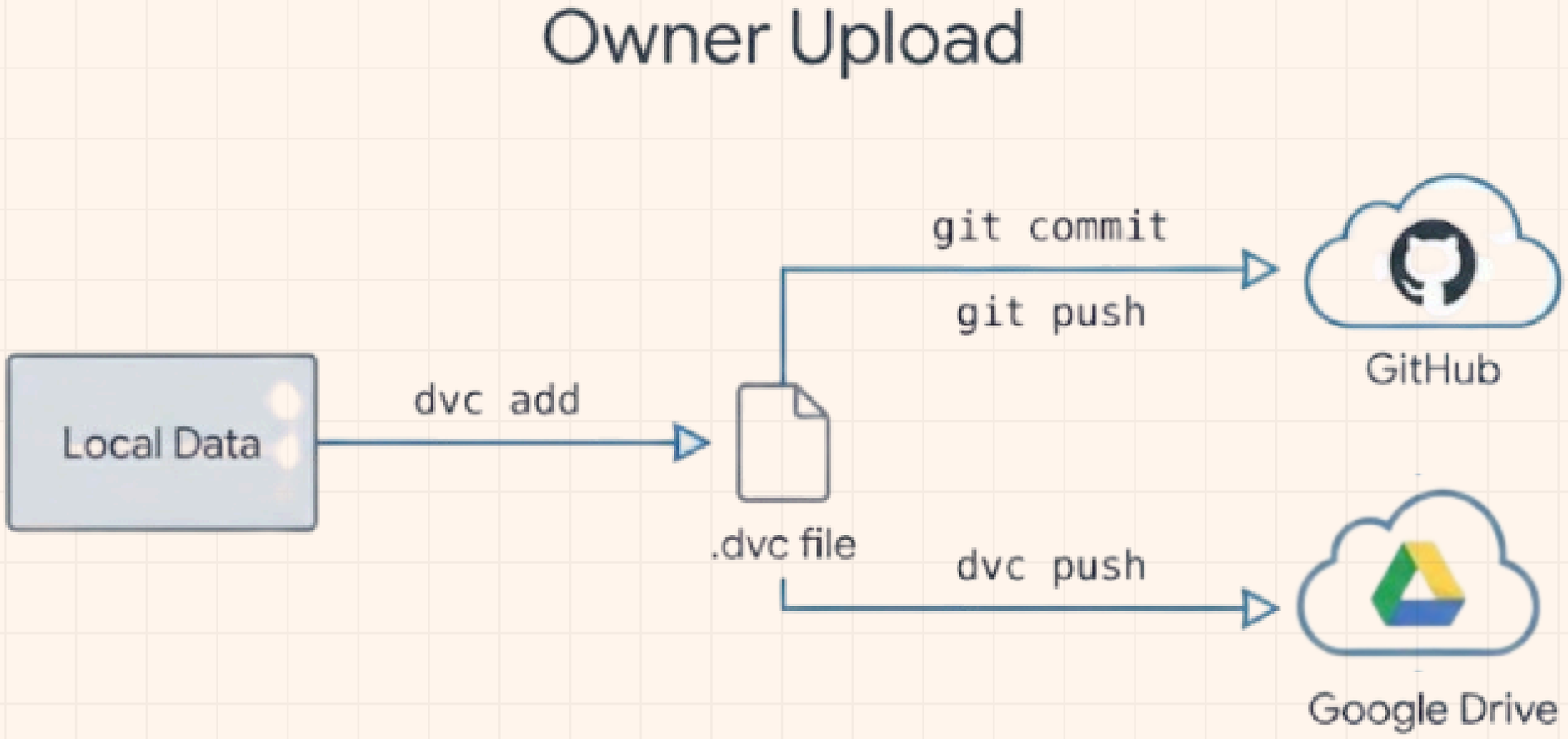
Git → .dvc files (metadata, 2KB)
DVC → Actual data (GBs) on Google Drive
GD Remote →

- **storage:** Main shared remote for approved data
- **uploads:** User-specific remote for personal uploads

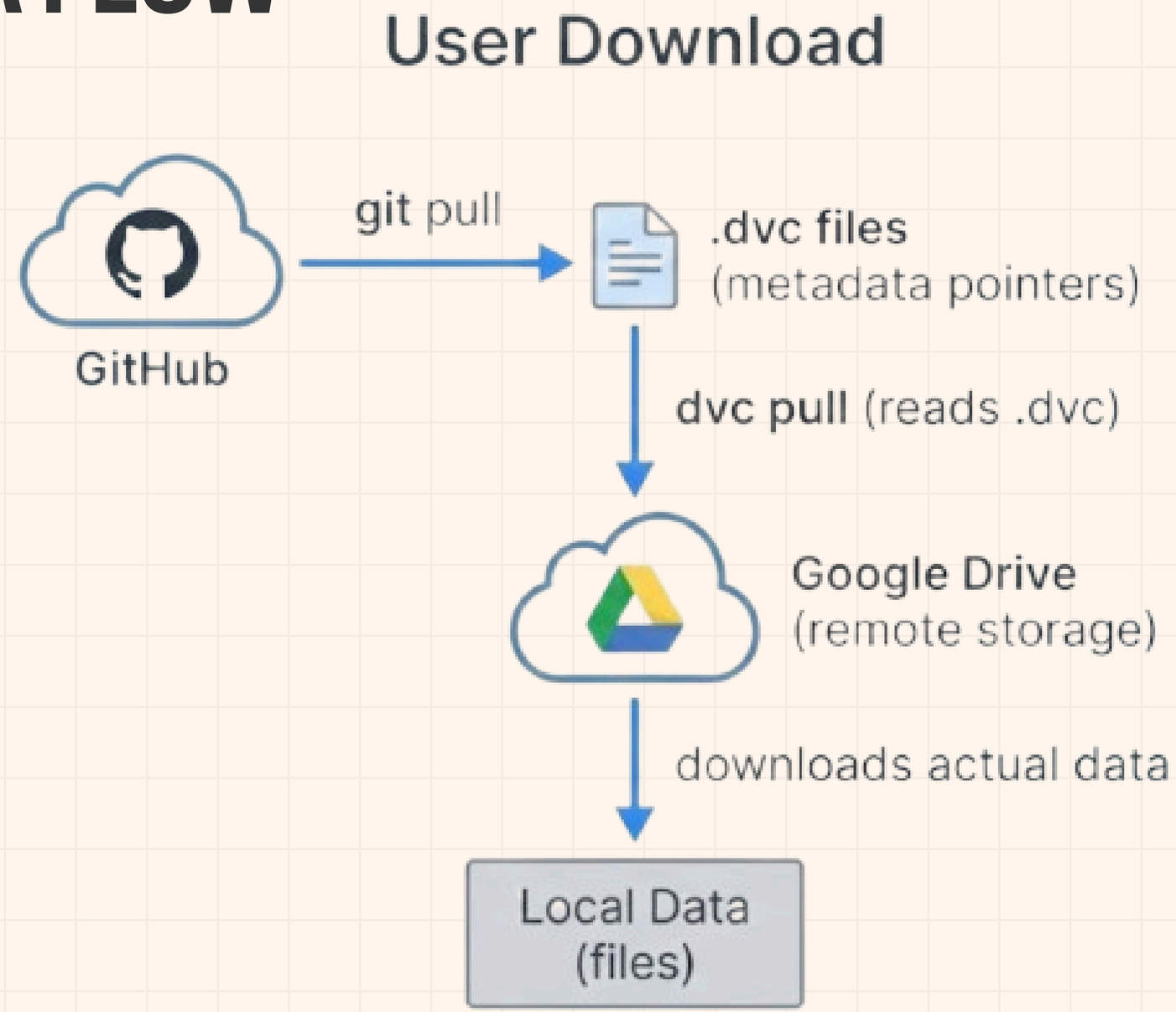
Result → Complete versioning, scalable storage, standard Git workflow

```
📄 Git Repository (GitHub)
├── Code files (.py, .md)
├── .dvc pointer files (tiny, ~1KB)
└── .gitignore (excludes actual data)

☁️ DVC Remote (Google Drive)
└── Actual data files (100+ GB)
    ├── IXI images (.nii.gz)
    ├── Segmentations
    └── Model outputs
```

5. **DATA FLOW**

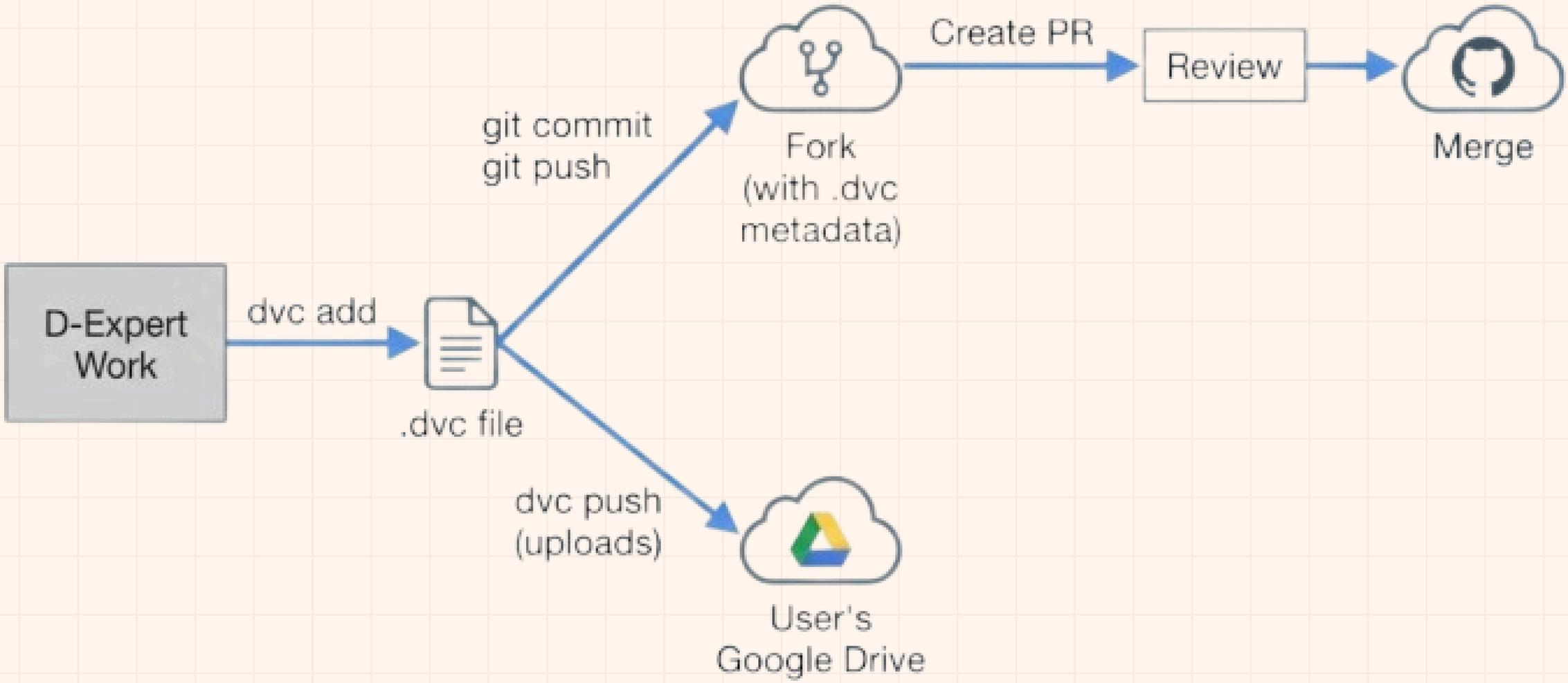

Owner Upload
Local Data
dvc add
.dvc file
git commit
git push
GitHub
dvc push
Google Drive

5. **DATA FLOW**


User Download
GitHub
git pull
.dvc files
(metadata pointers)
dvc pull (reads .dvc)
Google Drive
(remote storage)
downloads actual data
Local Data
(files)

5. **DATA FLOW**

User Contribution

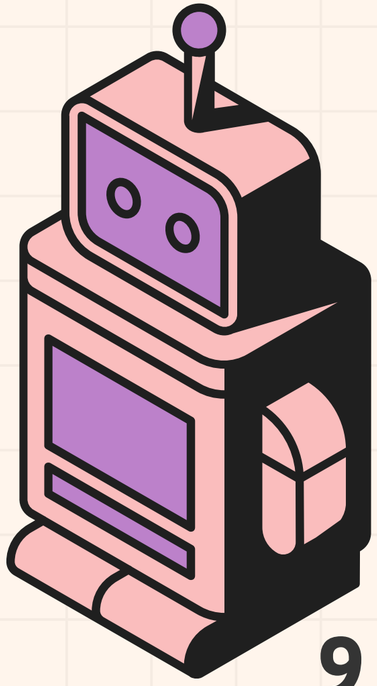


6. **VESSELVERSEOWNER.PY**

OWNER WORKFLOW

Command-line interface for dataset owners/maintainers to manage VesselVerse datasets

```
============================================================
          VesselVerse Dataset Owner Manager
============================================================


What would you like to do?

   [1] Initial Setup     - First time setup (configure credentials & download)
   [2] Update Dataset    - Sync latest changes from remote
   [3] Upload Dataset    - Push changes to the remote storage
   [4] Exit

Enter your choice [1-4]:
```

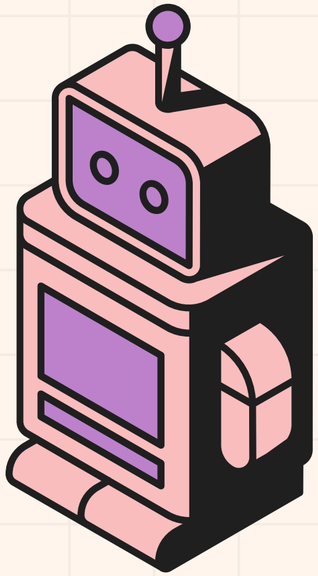


6. **VESSELVERSEUSER.PY**

USER WORKFLOW

Command-line interface for end users/contributors to download data and submit contributions.

```
============================================================
         VesselVerse Dataset User Manager
============================================================

What would you like to do?

  [1] Initial Setup       - First time setup (configure credentials & download)
  [2] Update Dataset      - Sync latest changes from remote
  [3] Upload Dataset      - Upload your modifications to the shared folder
  [4] Create Pull Request - Request owner review of your D-Expert data
  [5] Exit

Enter your choice [1-5]:
```



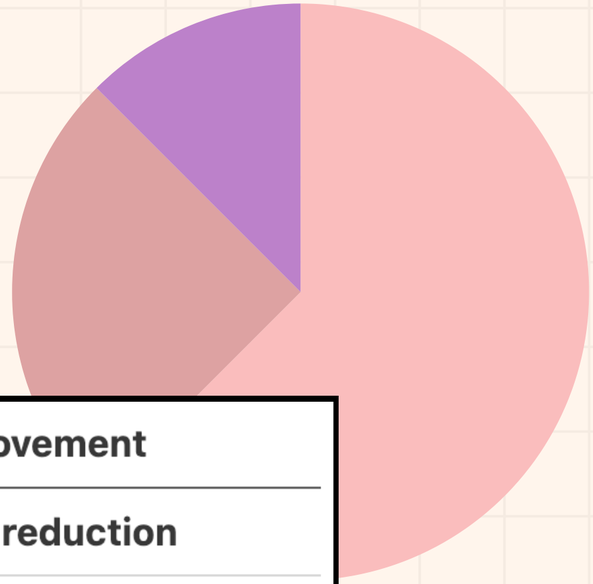
Difference Owner vs User:

- **Owner**: Full read + write access to all official datasets
- **User**: R + W access to D-Expert, read-only access to other datasets

7. COLLABORATION FLOW

```
1. 🍴 Fork Repositories
   • i-vesseg/VesselVerse-Framework → YourUsername/VesselVerse-Framework
   • i-vesseg/VesselVerse-Dataset → YourUsername/VesselVerse-Dataset

2. 💻 Setup Local Environment
   mkdir vesselverse && cd vesselverse
   git clone YourUsername/VesselVerse-Framework
   git clone YourUsername/VesselVerse-Dataset
   python3 vesselverseUser.py → [1] Initial Setup

3. 📊 Work on Data
   Create folders in D-Expert/
   Add your annotations, segmentations

4. ⬆️ Upload Work (vesselverseUser.py → [3])
   DVC push → Your Personal Google Drive
   Git push → Your Fork (with .dvc pointers)

5. 🔀 Submit for Review (vesselverseUser.py → [4])
   Creates PR: Your Fork → Official Repository
```

8. IMPACT & RESULTS



Quantified Benefits:

Metric	Before	After	Improvement
Setup Time	30+ minutes	5 minutes	**83% reduction**
Commands Required	15+ manual	3 menu selections	**80% reduction**
Prerequisites Knowledge	Git, DVC, GDrive	Run Python script	**No technical barrier**
PR Creation	Manual (GitHub expertise)	Automated (1 command)	**Fully automated**
Error Rate	High (manual typos)	Low (validated inputs)	**Significantly reduced**

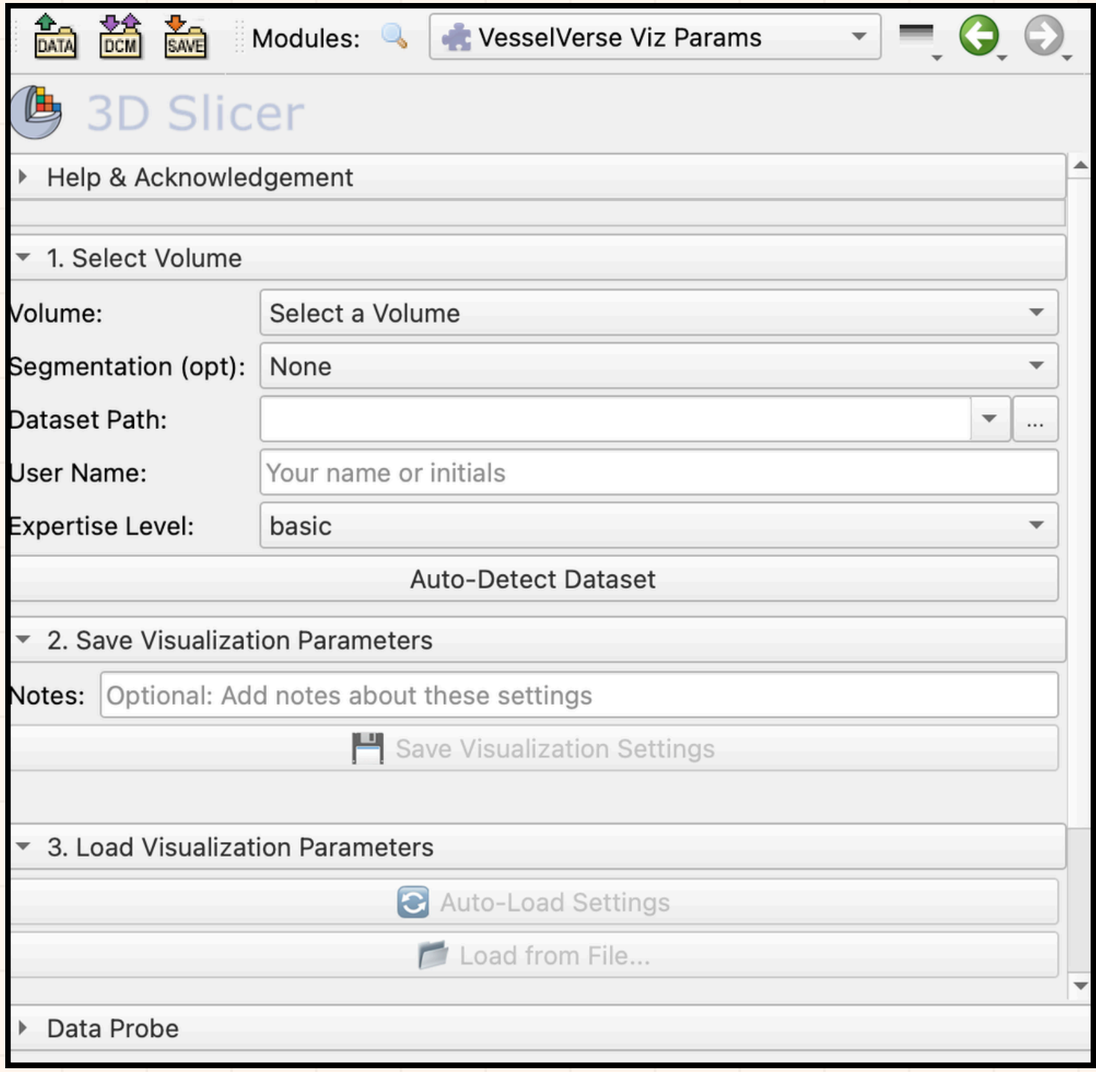
Qualitative Improvements:

- User-Friendly → Menu-driven interface
- Consistent → Same workflow for all datasets
- Maintainable → Centralized utility functions (vv_utils.py)- Shared configuration (config.py)
- Scalable → Independent DVC configurations per dataset

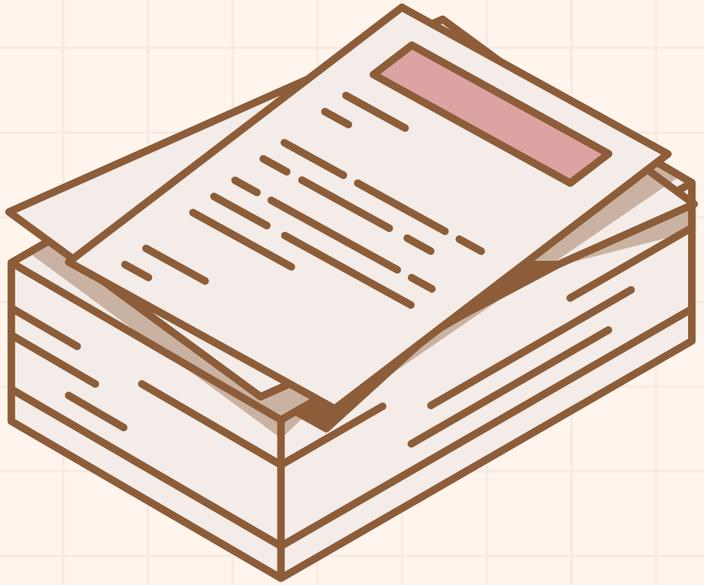
9. FUTURE DEVELOPMENTS

Visualization Parameters UI ⚡ In Progress

- **Problem:** Save/load visualization params in lightweight format
- **Current Solution:** Lightweight JSON
- **What I Built:**
  - 3D Slicer scripted UI module
  - Scripts for automated parameter management
  - Lightweight JSON format
- **Limitation:** Users must paste code into 3D Slicer's Python console to load the module (not user-friendly)
- **Next Step:** Make module installation one-click or package as official Slicer extension



10. **MAIN DOCUMENTATION**



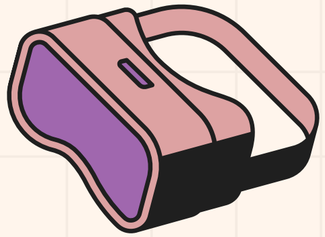
Main Documentation:

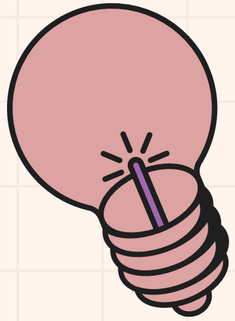
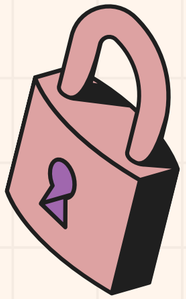
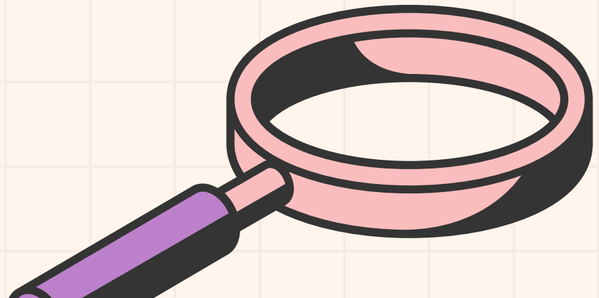
- **USER_MANUAL.md**
- **DEVELOPER_MANUAL.md**

Technical References:

- VesselVerse-Framework/docs/**viz_params_README.md**
- **VIZ_PARAMS.md**

GitRepo:
https://github.com/Alix03/TesterVesselVerse-Framework-Public





THANK YOU